Green DevOps – Energy Efficient CI/CD Pipeline

R26-SE-027

Project Proposal Report

I P Kodithuwakku – IT22189776

B.Sc. (Hons) Degree in Information Technology Specialized in Software Engineering

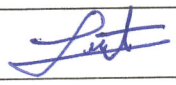
Department of Software Engineering

Sri Lanka Institute of Information Technology Sri Lanka

March 2026

DECLARATION

I declare that this is my own work, and this proposal does not knowingly incorporate any material previously submitted for a degree or diploma at any other university or higher education institution, nor does it contain any material that has been previously published or written by another person apart from where proper acknowledgment is given in the text.

Name	Student ID	Signature
Kodithuwakku I.P.	IT22189776	

The above candidate is conducting research for their undergraduate Dissertation under my supervision.

Name	Date	Signature
Mr.Jagath Wickramarathna	2026.03.11	

ABSTRACT

The rapid expansion of cloud computing and automated software delivery has significantly increased the energy consumption of digital infrastructure. Continuous Integration and Continuous Deployment (CI/CD) pipelines play a critical role in modern DevOps environments by enabling rapid software development, testing, and deployment. However, current DevOps monitoring tools primarily focus on performance, reliability, and resource utilization metrics, while providing limited visibility into the environmental impact of software delivery processes. This lack of sustainability monitoring represents a key research gap, particularly as organizations increasingly adopt environmental sustainability goals and seek to reduce their carbon footprint.

This research proposes a sustainability-aware monitoring framework for CI/CD pipelines within a Green DevOps architecture. The proposed system continuously collects operational metrics from pipeline executions, including CPU utilization, memory consumption, and execution duration. These metrics are used to estimate the energy consumption of CI/CD pipeline activities through proxy-based energy estimation models. Carbon emissions are then calculated by combining the estimated energy usage with carbon intensity data obtained from external sources. The monitoring component generates structured sustainability datasets that provide insights into the environmental impact of software delivery processes.

The proposed framework will be implemented using widely adopted DevOps technologies such as CI/CD platforms, Python-based monitoring tools, and cloud infrastructure services. The expected outcome is a monitoring framework that enables organizations to identify energy-intensive pipeline stages, optimize software delivery workflows, and make more environmentally responsible DevOps decisions. By integrating sustainability metrics into DevOps pipelines, this research contributes to the advancement of green software engineering practices and supports both environmental sustainability initiatives and emerging industry demand for sustainable digital infrastructure.

Keywords: Green DevOps, CI/CD Monitoring, Sustainable Software Engineering, Energy-Efficient Computing, Carbon-Aware Systems

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my supervisor, **Mr. Jagath Wickramaratna**, for his valuable guidance, constructive feedback, and continuous support throughout the preparation of this research proposal. His expertise, encouragement, and insightful recommendations greatly contributed to shaping the direction and quality of this work.

I would also like to extend my appreciation to my co-supervisor, **Ms. Madusha Weerasooriya**, for her helpful suggestions and continuous support during the development of this research proposal.

My sincere thanks also go to our external supervisor, **Mr. Dilanka Nayanajith**, for sharing his industry insights, professional advice, and valuable guidance, which helped us gain a better understanding of the practical aspects related to this research.

Furthermore, I would like to express my appreciation to the academic staff of the **Sri Lanka Institute of Information Technology (SLIIT)** for providing the knowledge, guidance, and resources that supported the successful preparation of this proposal.

Finally, I would like to thank my fellow team members **Mendis J.D.L**., **Thassara M.P.M**., and **Rathnayake D.M.B.H.** for their collaboration, constructive discussions, and teamwork during the early stages of this research.

I am also grateful to everyone who contributed directly or indirectly to the successful preparation of this research proposal.

TABLE OF CONTENT

DECLARATION 2
ABSTRACT 3
ACKNOWLEDGEMENT 4
TABLE OF CONTENT 5
LIST OF TABLES 6
LIST OF FIGURES 6
LIST OF ABBREVIATIONS 6
LIST OF APPENDICES 6
1. INTRODUCTION 7
2. OBJECTIVE 11
4. HIGH-LEVEL SYSTEM ARCHITECTURE 15
5. USER REQUIREMENTS 21
6. COMMERCIALIZATION PLAN 23
7. BUDGET AND JUSTIFICATION 27
8. WORK BREAKDOWN STRUCTURE (WBS) 29
9. GANTT CHART 30
10. REFFERENCES LIST 31
11. APPENDICES 32

LIST OF TABLES

Table 1: Budget Estimation.....27
Table 2: Work Breakdown Structure of the Monitor Component.....29

LIST OF FIGURES

Figure 1: High-Level Architecture of the Green DevOps Framework 16
Figure 2: Internal Architecture of the Monitor Component.....17
Figure 3: Persona 2 23
Figure 4: Persona 1 23
Figure 5: Gantt Chart of the Monitor Component 30

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
CI	Continuous Integration
CI/CD	Continuous Integration and Continuous Deployment
CPU	Central Processing Unit
DevOps	Development and Operations
IT	Information Technology
LCA	Life Cycle Assessment
ML	Machine Learning
SDG	Sustainable Development Goals

LIST OF APPENDICES

Appendix A: Example Monitoring Dataset Structure ..……..……32
Appendix B: Monitoring Script Example …….……..……32

1. INTRODUCTION

1.1 Background and Context

Modern software development increasingly relies on automated **Continuous Integration and Continuous Deployment (CI/CD)** pipelines to enable rapid and reliable software delivery. These pipelines automatically build, test, and deploy software applications whenever developers submit code changes. While CI/CD practices significantly improve development efficiency and reduce release cycles, they also require continuous computational resources and infrastructure support, particularly in cloud-based environments.

The rapid expansion of cloud computing and digital services has significantly increased the energy demand of data centers worldwide. According to the International Energy Agency (IEA), data centers and data transmission networks account for approximately **1–1.5% of global electricity consumption**, highlighting the growing environmental impact of digital infrastructure [8]. CI/CD pipelines contribute to this computational demand by repeatedly executing resource-intensive tasks such as compilation, automated testing, and container deployment throughout the software development lifecycle.

This issue affects multiple stakeholders within the software ecosystem. Software development organizations rely heavily on CI/CD pipelines to maintain continuous delivery practices, while cloud service providers must allocate the necessary computing resources to execute these workflows. As organizations increasingly adopt sustainability initiatives and environmental responsibility goals, understanding and managing the environmental impact of software development processes has become an important concern.

Although several DevOps monitoring tools exist today, most focus primarily on operational metrics such as system performance, execution time, and failure rates. These tools provide limited visibility into the **energy consumption and carbon emissions associated with CI/CD pipeline executions**. Recent research has highlighted the environmental impact of software delivery pipelines and emphasized the need for sustainability-aware software engineering practices [6]. However, practical solutions for monitoring and analyzing sustainability metrics within CI/CD pipelines remain limited.

To address this challenge, this research proposes a **sustainability-aware monitoring framework for CI/CD pipelines** that estimates energy consumption and carbon emissions based on pipeline execution metrics. By providing visibility into the environmental impact of software delivery processes, the proposed system aims to support more sustainable DevOps practices. This research also aligns with the **United Nations Sustainable Development Goals**, particularly **SDG 12 (Responsible Consumption and Production)** and **SDG 13 (Climate Action)**, by promoting more efficient use of computational resources and reducing the environmental footprint of digital systems.

1.2 Literature Review

Modern software development increasingly relies on Continuous Integration and Continuous Deployment (CI/CD) pipelines to automate the building, testing, and deployment of software systems. These pipelines enable rapid and reliable software delivery by automatically executing workflows whenever developers commit changes to a repository. However, CI/CD pipelines frequently execute resource-intensive tasks such as compiling code, executing automated test suites, and deploying applications across distributed cloud infrastructure. As a result, the continuous operation of CI/CD pipelines contributes to growing energy consumption in cloud computing environments and data centers [8].

With the expansion of digital services and cloud infrastructure, the environmental impact of software systems has become an important concern within the field of sustainable computing. Sustainable software engineering focuses on designing, developing, and operating software systems in ways that minimize energy consumption and carbon emissions while maintaining performance and reliability. Hindle highlights that software systems indirectly influence energy consumption through the hardware they operate on, emphasizing the need for energy-aware software design practices [6]. As software systems scale across large cloud infrastructures, the cumulative environmental impact of software execution becomes increasingly significant.

Recent research has explored the integration of sustainability principles into DevOps practices. DevOps aims to improve collaboration between development and operations teams while enabling faster and more reliable software delivery through automation. However, traditional DevOps pipelines primarily optimize for speed, reliability, and scalability rather than environmental sustainability. Ailane et al. proposed a Green DevOps framework that incorporates sustainability considerations into different phases of the DevOps lifecycle, including planning, development, testing, and operations [1]. Their work demonstrates how organizations can integrate environmental sustainability into DevOps workflows, although the framework primarily provides conceptual guidance rather than practical mechanisms for measuring sustainability metrics within CI/CD pipelines.

Another area of research investigates the environmental impact of CI/CD pipelines themselves. Saavedra et al. conducted a large-scale empirical study analyzing thousands of GitHub Actions workflows to estimate the carbon and water footprint of CI/CD activities [3]. Their findings show that automated build and testing processes executed across large software ecosystems can generate significant environmental impact. While the study provides valuable insights into the scale of the problem, it focuses on large-scale measurement rather than providing a monitoring system that developers can directly use within their own CI/CD pipelines.

Researchers have also explored carbon-aware computing strategies aimed at reducing emissions by scheduling computational workloads when the carbon intensity of electricity generation is lower. Claßen et al. proposed a carbon-aware CI/CD architecture that schedules pipeline executions based on carbon intensity forecasts and historical workflow execution patterns [2]. Their results indicate that adjusting pipeline execution times can significantly reduce emissions without disrupting developer productivity. However, such scheduling mechanisms require detailed monitoring data about pipeline execution patterns and energy consumption, which is not typically available in standard DevOps monitoring tools.

Several studies have also investigated integrating energy measurement mechanisms into CI/CD pipelines. Aneggi et al. introduced an energy-aware continuous integration framework that measures the energy consumption of software builds and test executions during pipeline runs [5]. This approach enables developers to observe the energy impact of code changes and software updates. Nevertheless, direct energy measurement approaches often require specialized hardware instrumentation or infrastructure-level monitoring capabilities, which are difficult to implement in many cloud-based CI/CD environments.

Research on software frugality in continuous integration environments further highlights the environmental impact of repeated CI executions. Perez et al. demonstrated that although the energy consumption of individual CI runs may appear small, frequent pipeline executions can accumulate significant energy consumption over time [4]. Their work emphasizes the need for monitoring systems that can provide developers with visibility into the environmental implications of their CI/CD workflows.

Beyond CI/CD-specific studies, broader research on sustainable computing and green cloud infrastructure also contributes to this domain. Beloglazov and Buyya explored energy-efficient resource management techniques in cloud computing environments, highlighting how intelligent workload management can significantly reduce energy consumption in large-scale data centers [7]. Similarly, the Green Software Foundation introduced the Software Carbon Intensity (SCI) specification to provide standardized methods for measuring and reporting the carbon footprint of software systems [9].

Despite these advances, several limitations remain in existing research. Many current approaches focus on infrastructure-level optimization or carbon-aware scheduling rather than providing detailed monitoring at the CI/CD pipeline stage level. Existing DevOps monitoring tools primarily measure performance metrics such as execution time, system utilization, and failure rates but do not translate these metrics into energy consumption or carbon emission indicators. Furthermore, monitoring data generated by CI/CD pipelines is rarely structured into reusable datasets that support long-term analysis, predictive modeling, or optimization of sustainable DevOps practices.

These limitations highlight the need for a sustainability-aware monitoring framework capable of collecting CI/CD execution metrics, estimating energy consumption and carbon emissions, and structuring the data into reusable sustainability datasets. Such a monitoring system can provide developers and organizations with actionable insights into the environmental impact of their software delivery pipelines while enabling future optimization strategies within Green DevOps environments.

1.3 Research Gap

Although significant research has been conducted in the area of sustainable software engineering and Green DevOps, several limitations remain in the current body of work. Existing studies primarily focus on integrating sustainability principles into DevOps practices at a conceptual level. For example, Ailane et al. proposed a Green DevOps framework that introduces sustainability considerations throughout the DevOps lifecycle [1]. However, their work mainly provides strategic guidelines and does not present a practical monitoring mechanism for measuring sustainability metrics within CI/CD pipelines.

Several researchers have also investigated the environmental impact of CI/CD pipelines themselves. Saavedra et al. performed a large-scale empirical analysis of CI/CD workflows using GitHub Actions data and demonstrated that CI/CD activities can generate a significant carbon footprint across large software ecosystems [3]. While this research highlights the magnitude of the environmental impact, it focuses primarily on impact estimation rather than providing tools that developers can use to monitor sustainability metrics within their own CI/CD pipelines.

Other research efforts have explored carbon-aware computing approaches to reduce emissions in software systems. Claßen et al. proposed a carbon-aware CI/CD architecture that schedules pipeline executions according to carbon intensity forecasts [2]. Although this approach can reduce emissions, it relies heavily on accurate monitoring data about pipeline execution patterns and resource usage, which is not typically available in existing DevOps monitoring systems.

Similarly, studies on energy-aware CI/CD pipelines have explored methods for measuring energy consumption during software builds and testing processes [5]. However, many of these approaches require specialized hardware instrumentation or infrastructure-level monitoring capabilities, making them difficult to implement in cloud-based development environments where developers have limited control over underlying hardware resources.

Furthermore, current DevOps monitoring tools primarily focus on performance metrics such as execution time, failure rates, and resource utilization. These metrics are rarely translated into energy consumption or carbon emission indicators that can support sustainability-oriented decision-making. In addition, monitoring data generated during CI/CD executions is typically not structured into datasets that can support long-term analysis, predictive modeling, or optimization of sustainable DevOps strategies [4].

Therefore, there is a need for a sustainability-aware monitoring framework that can collect CI/CD pipeline execution metrics, estimate energy consumption and carbon emissions, and transform the collected data into structured sustainability datasets. Such a monitoring system would provide developers and organizations with detailed visibility into the environmental impact of their software delivery processes while enabling future optimization strategies for Green DevOps environments.

2. OBJECTIVE

2.1 Main Objective

To design a sustainability-aware monitoring component that collects CI/CD execution metrics, estimates energy consumption and carbon emissions, and structures the data into reusable sustainability datasets.

2.2 Specific Objective

1. Develop a sustainability-aware monitoring component

   To design and implement a monitoring module that collects CI/CD pipeline execution metrics and estimates energy consumption and carbon emissions for each pipeline stage.

2. Design energy-efficient CI/CD deployment strategies

   To develop deployment and infrastructure configurations that reduce resource usage and improve energy efficiency during software deployment processes.

3. Implement operational optimization mechanisms

   To analyze pipeline execution patterns and implement operational improvements that minimize unnecessary resource consumption in CI/CD environments.

4. Evaluate the effectiveness of the Green DevOps framework

   To experimentally evaluate the proposed framework by measuring improvements in energy efficiency and carbon footprint within a controlled CI/CD pipeline environment.

3. METHODOLOGY

This research adopts a **Design Science Research (DSR)** approach to design, implement, and evaluate a sustainability-aware monitoring framework for CI/CD pipelines. Design Science Research is widely used in software engineering research where new artifacts such as systems, models, or frameworks are developed to address identified problems. In this study, the artifact is a **Green DevOps monitoring framework that measures and analyzes the environmental impact of CI/CD workflows**.

3.1 Research Approach

The study follows a Design Science Research methodology consisting of the following stages:

1. Problem identification and literature analysis
2. Design and development of the monitoring framework
3. Implementation within a CI/CD pipeline environment
4. Experimental evaluation and validation

This approach allows the research to both develop a practical monitoring system and evaluate its effectiveness in measuring sustainability metrics in CI/CD pipelines.

3.2 CI/CD Environment and System Implementation

A CI/CD pipeline environment will be implemented using widely adopted DevOps platforms such as **GitHub Actions or Jenkins**. These platforms are selected because they are commonly used in modern software development and provide programmable workflows that allow integration of monitoring scripts.

A sample application repository will be used to simulate real development workflows including code compilation, automated testing, and deployment stages. This environment enables controlled experimentation and provides realistic pipeline execution data.

During the early stages of system development, optimization components within the Green DevOps framework may utilize **existing or publicly available datasets** to simulate sustainability metrics. This allows different components of the system to be developed in parallel without dependency on the monitoring module. As the monitoring component becomes operational, **real sustainability data collected from CI/CD pipeline executions will gradually replace these external datasets**, enabling full system integration and more realistic evaluation.

3.3 Monitoring Data Collection

A sustainability-aware monitoring component will be integrated into the CI/CD pipeline to collect execution and resource utilization metrics during pipeline runs. The monitoring system will record metrics such as:

• Pipeline stage execution time
• CPU utilization
• Memory consumption
• Job execution metadata

These metrics form the basis for estimating the energy consumption of CI/CD pipeline activities and generating sustainability datasets for further analysis.

3.4 Energy and Carbon Estimation Methods

Energy consumption will be estimated using **proxy-based energy estimation models**, which approximate energy usage based on resource utilization metrics. A simplified estimation model will be used where energy consumption is derived from CPU utilization and execution duration.

Estimated carbon emissions will then be calculated by combining the energy consumption values with **carbon intensity data obtained from publicly available datasets** such as ElectricityMaps. This method enables estimation of the carbon footprint associated with CI/CD pipeline executions.

3.5 Tools and Platforms

The implementation of the monitoring framework will utilize widely available open-source tools to ensure practicality and reproducibility. The CI/CD pipelines will be implemented using **GitHub Actions or Jenkins**, while monitoring scripts will be developed using **Python** to collect and process pipeline metrics. Carbon intensity data will be obtained from public APIs such as **ElectricityMaps**. Visualization and reporting of sustainability metrics may be implemented using monitoring dashboards such as **Grafana**.

These tools are selected due to their widespread adoption in DevOps environments and their flexibility for integrating custom monitoring functionality.

3.6 Data Handling and Ethical Considerations

The research does not involve human participants or personal data. All collected data will consist of system-generated metrics related to CI/CD pipeline execution and resource utilization. Therefore, the study does not require ethical clearance involving human subjects.

Additionally, only publicly available carbon intensity datasets and synthetic CI/CD workflows will be used for experimentation, ensuring that no confidential or sensitive organizational data is collected.

3.7 Validation and Evaluation Strategy

The effectiveness of the proposed monitoring framework will be evaluated through controlled experiments using the implemented CI/CD pipeline environment. Multiple pipeline executions will be analyzed to measure sustainability metrics and identify energy-intensive pipeline stages.

The evaluation will focus on the following performance indicators:

- Estimated energy consumption per pipeline execution
- Carbon emissions associated with CI/CD workflows
- Resource utilization patterns across pipeline stages
- Ability of the monitoring system to provide actionable sustainability insights

The evaluation will also compare results obtained using **external datasets during early development stages** with results derived from **real sustainability data generated by the monitoring component**. This comparison will help assess the accuracy and practical usefulness of the proposed monitoring framework in supporting sustainable DevOps decision-making.

4. HIGH-LEVEL SYSTEM ARCHITECTURE

4.1 Overview of the Proposed System

The proposed system introduces sustainability awareness into the DevOps lifecycle by integrating energy and carbon monitoring within CI/CD pipelines. While traditional DevOps practices focus on rapid software delivery, reliability, and scalability, they typically lack mechanisms for measuring the environmental impact of software delivery processes such as energy consumption and carbon emissions.

To address this limitation, the proposed Green DevOps framework incorporates a **sustainability-aware monitoring component** that continuously collects and analyzes CI/CD pipeline execution data. The monitoring component acts as the central sustainability data provider within the system, transforming operational metrics into energy and carbon indicators that can support environmentally informed decision-making.

The overall architecture includes several components that operate within the CI/CD lifecycle. The **Release component** focuses on improving release preparation processes using sustainability insights, the **Deploy component** manages application deployment while considering resource efficiency, and the **Operate component** monitors deployed applications during runtime and performs infrastructure optimization. These components utilize sustainability data generated by the monitoring component to guide their decision-making processes.

The **Monitoring component** forms the core sustainability measurement layer of the system. It collects pipeline execution metrics and infrastructure resource utilization data, estimates energy consumption using proxy-based models, and calculates carbon emissions using carbon intensity data from external sources. The processed information is then structured into sustainability datasets that provide visibility into the environmental impact of CI/CD pipeline executions.

By integrating this monitoring capability into the DevOps pipeline, the system establishes a **sustainability feedback loop** where operational data collected from CI/CD executions can influence future release scheduling, deployment strategies, and infrastructure management decisions.

4.2 System Architecture Diagram

The overall architecture of the proposed Green DevOps framework is illustrated in Figure X. The diagram presents the interaction between the Release, Deploy, Operate, and Monitoring components within the CI/CD pipeline. While all components contribute to sustainability-aware DevOps practices, the monitoring component acts as the central layer responsible for collecting sustainability metrics and generating datasets used by the other components.

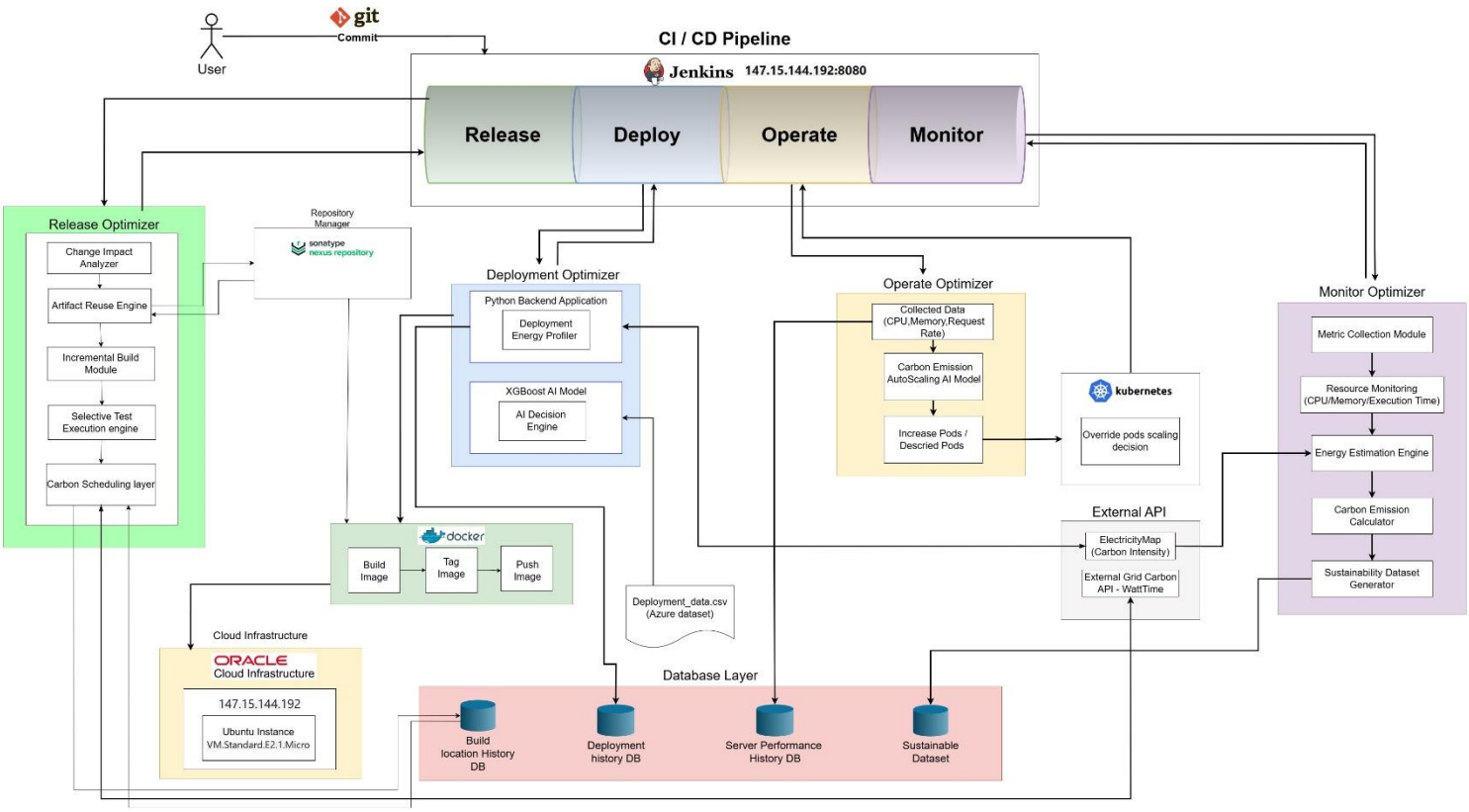


Figure 1: High-Level Architecture of the Green DevOps Framework

4.3 Monitoring Component Architecture

The Monitoring component forms the sustainability measurement layer of the proposed Green DevOps framework. Its primary objective is to collect CI/CD pipeline execution metrics, estimate the energy consumption of pipeline activities, and calculate the associated carbon emissions. The monitoring component transforms raw operational data into structured sustainability datasets that can support optimization decisions in other components of the system.

Figure **2** illustrates the internal architecture of the monitoring component, including the metric collection module, energy estimation module, carbon estimation module, and sustainability dataset generator

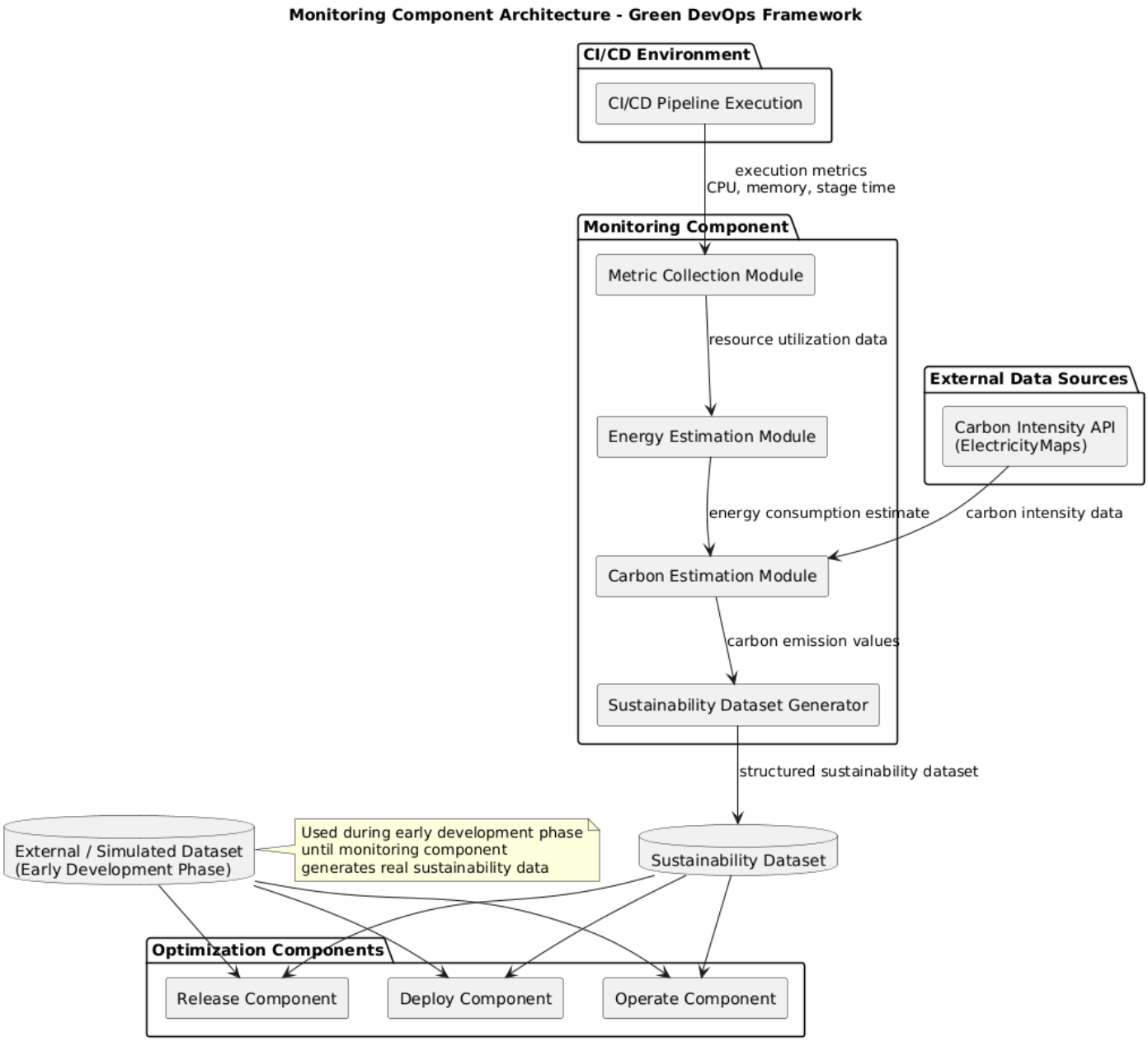


Figure 2: Internal Architecture of the Monitor Component

The architecture of the monitoring component consists of several internal modules that operate sequentially to process and analyze pipeline execution data.

Metric Collection Module

The Metric Collection Module gathers raw operational data from CI/CD pipeline executions and the underlying infrastructure environment. Monitoring scripts integrated into the pipeline collect metrics such as pipeline stage execution time, CPU utilization, memory consumption, and job metadata. These metrics represent the primary inputs required for sustainability analysis.

Energy Estimation Module

The Energy Estimation Module processes the collected resource utilization metrics to estimate the energy consumption associated with CI/CD pipeline activities. Since direct energy measurement is often not available in cloud-based environments, proxy-based energy estimation models are used to approximate energy usage based on CPU utilization and execution duration.

Carbon Estimation Module

The Carbon Estimation Module calculates the estimated carbon emissions of pipeline executions by combining the energy consumption estimates with carbon intensity data obtained from external sources such as ElectricityMaps. This enables the monitoring system to estimate the environmental impact of CI/CD operations.

Sustainability Dataset Generator

The Sustainability Dataset Generator structures the processed monitoring data into a dataset that contains both raw execution metrics and calculated sustainability indicators. The generated dataset includes parameters such as pipeline stage information, resource utilization metrics, estimated energy consumption, and carbon emission values. This dataset can then be used by other system components or analytical modules to support sustainability optimization.

Together, these modules enable the monitoring component to convert operational pipeline data into meaningful sustainability metrics that can guide environmentally responsible DevOps practices.

4.4 Component Interactions

The Monitoring component interacts with the other components of the Green DevOps framework by providing sustainability-related insights derived from CI/CD pipeline execution data.

During the early development phase of the system, other components may utilize existing or simulated datasets to perform optimization tasks. Once the monitoring component becomes operational, these datasets are gradually replaced with real sustainability metrics collected directly from CI/CD pipeline executions.

The interaction between components can therefore be summarized as follows:

• The Monitoring component collects operational metrics from CI/CD pipeline executions.
• The processed sustainability metrics are shared with the **Release component** to support carbon-aware release scheduling and workload preparation.
• The **Deploy component** utilizes the monitoring insights to select energy-efficient deployment strategies.
• The **Operate component** uses the sustainability data to optimize runtime resource allocation and infrastructure scaling.

This interaction establishes a sustainability feedback mechanism where monitoring data collected from system operations influences future DevOps decisions.

4.5 Data Flow

The data flow within the monitoring component begins with the execution of CI/CD pipeline stages. During each pipeline execution, monitoring scripts collect execution and infrastructure metrics such as CPU usage, memory consumption, and stage execution duration.

These metrics are processed by the **Energy Estimation Module**, which applies proxy-based energy estimation models to approximate the energy consumption associated with pipeline operations. The estimated energy values are then passed to the **Carbon Estimation Module**, where they are combined with carbon intensity data obtained from external sources such as ElectricityMaps to estimate carbon emissions.

The processed information is structured by the **Sustainability Dataset Generator** into a dataset containing both raw execution metrics and calculated sustainability indicators. This dataset provides visibility into the environmental impact of CI/CD pipeline activities and can be consumed by other components of the system to support sustainability-aware optimization and decision-making processes.

4.6 Scalability and Security Considerations

The monitoring architecture is designed to operate efficiently within modern cloud-based CI/CD environments. The monitoring modules are lightweight and integrated directly into the pipeline workflow, ensuring minimal performance overhead during pipeline execution. The system architecture can scale with increasing pipeline workloads by collecting monitoring metrics asynchronously and performing sustainability calculations independently from critical pipeline tasks.

From a security perspective, the monitoring system collects only system-level operational metrics and does not access sensitive application data or user information. All collected data consists of infrastructure and execution metrics generated during CI/CD operations. Additionally, external data sources such as carbon intensity APIs are accessed through secure communication protocols to ensure the integrity and reliability of sustainability calculations.

5. USER REQUIREMENTS

5.1 Stakeholder Identification

Several stakeholders interact with or benefit from the proposed sustainability-aware monitoring system within the Green DevOps framework.

DevOps Engineers
DevOps engineers are responsible for managing CI/CD pipelines and infrastructure. They require visibility into the energy consumption and carbon emissions associated with pipeline executions in order to optimize pipeline configurations and infrastructure usage.

Software Developers
Developers rely on CI/CD pipelines for automated builds, testing, and deployment. Sustainability insights generated by the monitoring component help developers understand the environmental impact of their software development activities.

System Administrators / Cloud Engineers
Infrastructure administrators manage cloud resources used by CI/CD pipelines. The monitoring system provides resource utilization and sustainability metrics that support more efficient infrastructure management.

Organizations with Sustainability Goals
Many organizations aim to reduce their environmental footprint and meet corporate sustainability targets. The monitoring framework provides measurable indicators of energy consumption and carbon emissions associated with software delivery processes.

5.2 Functional Requirements

The functional requirements define the key capabilities that the monitoring component must provide in order to support sustainability-aware DevOps operations.

FR1 – Metric Collection
The system shall collect CI/CD pipeline execution metrics such as stage execution time, CPU utilization, memory usage, and job metadata during pipeline runs.

FR2 – Energy Consumption Estimation
The system shall estimate the energy consumption of CI/CD pipeline executions using proxy-based energy estimation models derived from collected resource utilization metrics.

FR3 – Carbon Emission Estimation
The system shall calculate estimated carbon emissions associated with pipeline executions by combining energy consumption estimates with carbon intensity data obtained from external sources.

FR4 – Sustainability Dataset Generation
The system shall generate structured sustainability datasets containing pipeline execution metrics, estimated energy consumption values, and calculated carbon emission indicators.

FR5 – Integration with DevOps Components
The system shall provide sustainability metrics that can be accessed by other components of the Green DevOps framework to support optimization decisions related to release scheduling, deployment strategies, and infrastructure operations.

FR6 – Historical Monitoring Data Storage
The system shall store sustainability metrics from multiple pipeline executions to enable analysis of pipeline behavior and identification of energy-intensive stages.

5.3 Non-Functional Requirements

The non-functional requirements describe quality attributes that ensure the monitoring component operates effectively within CI/CD environments.

NFR1 – Performance
The monitoring component shall collect and process pipeline metrics with minimal impact on CI/CD pipeline execution time to avoid disrupting software delivery workflows.

NFR2 – Scalability
The monitoring system shall support monitoring of multiple CI/CD pipeline executions across different projects and environments without significant degradation in performance.

NFR3 – Security
The monitoring component shall collect only system-level operational metrics and must not access or store sensitive application code or confidential user information.

NFR4 – Reliability
The monitoring system shall consistently capture pipeline execution metrics and generate sustainability datasets without data loss during pipeline execution.

NFR5 – Interoperability
The monitoring component shall integrate with commonly used CI/CD platforms such as GitHub Actions or Jenkins to ensure compatibility with modern DevOps environments.

6. COMMERCIALIZATION PLAN

6.1 Target Market and Customer Persona

The proposed Green DevOps monitoring framework targets organizations that rely heavily on automated CI/CD pipelines and cloud infrastructure for software delivery. As companies increasingly adopt sustainability initiatives and environmental reporting requirements, tools that provide visibility into the environmental impact of software development processes are becoming valuable.

The primary target markets include:

- Software development companies using cloud-based CI/CD pipelines
- Technology startups operating cloud-native applications
- Enterprises with sustainability reporting requirements
- Cloud infrastructure and DevOps platform providers

Two key customer personas have been identified for the proposed solution.

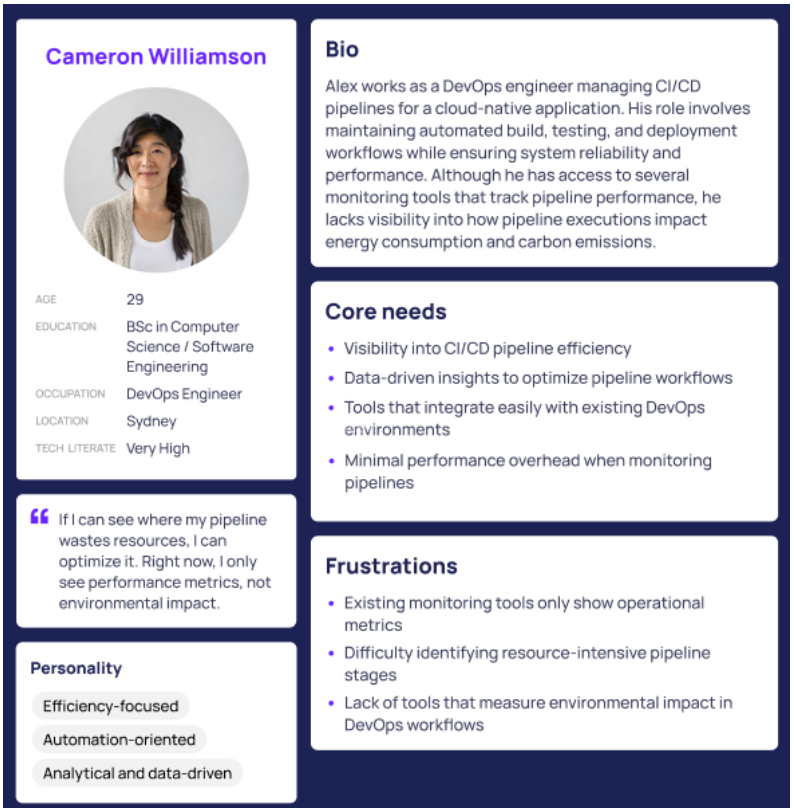


Figure 4: Persona 1

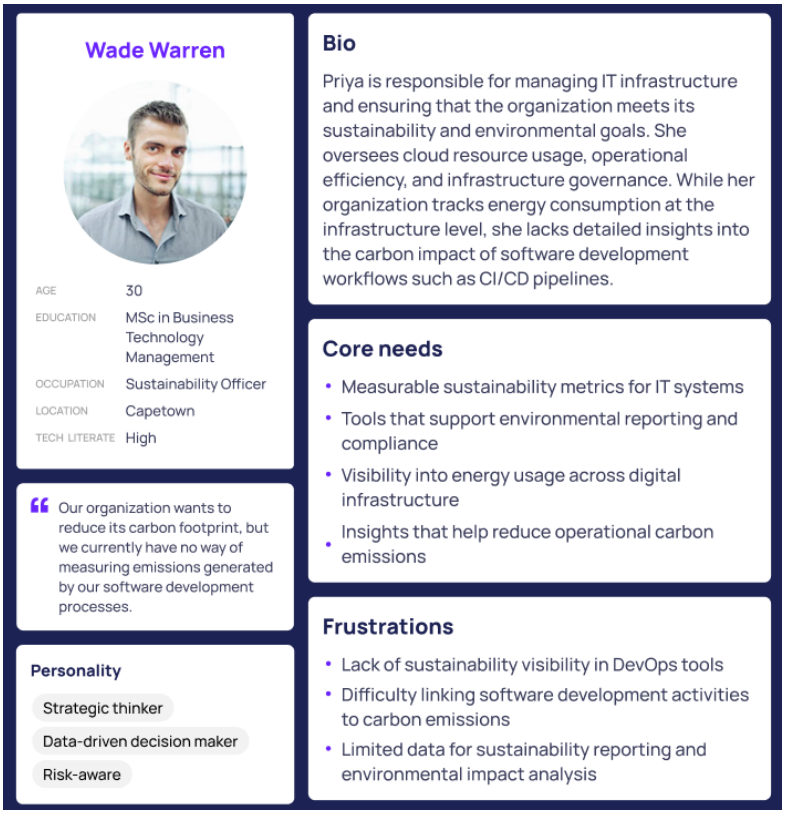


Figure 3: Persona 2

6.2 Value Proposition

The proposed system provides value by introducing **sustainability visibility into CI/CD pipelines**, an area that is largely overlooked by existing DevOps monitoring tools.

Currently, most DevOps monitoring systems focus on operational metrics such as system performance, failure rates, and resource utilization. However, they do not provide direct insights into the **environmental impact of software delivery processes**.

The proposed monitoring framework addresses this gap by:

- Measuring **energy consumption associated with CI/CD pipeline executions**
- Estimating **carbon emissions generated during software delivery**
- Providing **structured sustainability datasets** that can support optimization decisions

By enabling organizations to measure and analyze the environmental impact of their DevOps pipelines, the system supports both **operational efficiency and sustainability goals**. This helps organizations reduce unnecessary resource consumption, optimize infrastructure usage, and meet corporate environmental responsibility targets.

6.3 Revenue Model

The proposed system can be commercialized through several revenue models.

Subscription-Based SaaS Model

The monitoring system can be offered as a cloud-based software service where organizations subscribe to the platform to monitor sustainability metrics within their CI/CD pipelines. Pricing tiers can be based on factors such as the number of monitored pipelines, data volume, or infrastructure scale.

Enterprise Licensing

Large organizations may deploy the monitoring system within their internal infrastructure environments. In this case, enterprise licensing can provide access to the monitoring platform along with integration support and advanced analytics features.

DevOps Platform Integration

The monitoring framework could also be commercialized as a plugin or extension for popular DevOps platforms such as GitHub Actions, GitLab CI, or Jenkins, allowing organizations to integrate sustainability monitoring directly into their existing development environments.

6.4 Cost Estimation

The primary costs associated with commercializing the proposed system include development, infrastructure, and operational expenses.

Development Costs

Development involves designing the monitoring modules, integrating CI/CD data collection mechanisms, implementing energy and carbon estimation models, and developing analytics dashboards.

Estimated development cost includes:

- Software development effort
- system integration and testing
- UI/dashboard development

Infrastructure Costs

Since the system processes monitoring data and stores sustainability datasets, cloud infrastructure will be required for data processing and storage.

Estimated infrastructure costs include:

- cloud computing resources
- data storage services
- API access for carbon intensity data

These costs can scale depending on the number of organizations or CI/CD pipelines monitored by the platform.

6.5 Pricing Strategy

The proposed pricing strategy follows a **tiered subscription model** that accommodates organizations of different sizes.

Example pricing tiers could include:

- **Starter Tier** – Monitoring for small teams with limited CI/CD pipelines
- **Professional Tier** – Monitoring for mid-sized development teams with multiple pipelines
- **Enterprise Tier** – Advanced sustainability analytics and enterprise integration capabilities

Pricing can be based on metrics such as the number of monitored pipelines, data volume, or number of users.

6.6 Competitive Advantage

The proposed monitoring framework provides several advantages compared to existing DevOps monitoring tools.

First, most current DevOps monitoring platforms focus on operational metrics such as performance monitoring, system availability, and resource usage. They rarely provide insights into **energy consumption or carbon emissions associated with CI/CD pipeline executions**.

Second, existing sustainability tools typically focus on infrastructure-level emissions, such as overall cloud resource usage. They do not provide detailed visibility into the environmental impact of individual CI/CD pipeline stages.

The proposed system addresses these limitations by providing **pipeline-level sustainability monitoring**, allowing organizations to identify energy-intensive pipeline stages and optimize their software delivery processes accordingly.

Additionally, the monitoring framework generates **structured sustainability datasets** that can support advanced analytics, optimization algorithms, and sustainability reporting.

6.7 Intellectual Property Considerations

The proposed system includes several elements that could be protected through intellectual property mechanisms.

The architecture of the sustainability-aware monitoring framework, including the integration of CI/CD pipeline metrics with energy and carbon estimation models, may be eligible for intellectual property protection. Additionally, proprietary algorithms used for energy estimation, carbon calculation, and sustainability dataset generation could be protected through patents or trade secrets.

The software implementation of the monitoring framework may also be protected through copyright as a software product. If the system is commercialized as a platform or plugin, licensing agreements and intellectual property protections can ensure that the technology remains proprietary.

7. BUDGET AND JUSTIFICATION

7.1 Budget Estimation

Table 1: Budget Estimation

Category	Item	Quantity	Estimated Price (LKR)
Hardware	Raspberry Pi (Monitoring Hardware Platform)	1	20,000.00
	DHT22 Temperature Sensor	1	1,000.00
	MicroSD Card (32GB)	1	2,000.00
	Power Supply & Cables	1 Set	1,500.00
Cloud Services	Oracle Cloud Infrastructure (Compute & Storage)	1 Year	Free Tier
Software	Open-source tools (Python, pandas, psutil, XGBoost)	N/A	0.00
	CI/CD Tools (GitHub Actions / Jenkins)	N/A	Free Tier
Human Effort	Development & Research Team (5 Students)	Project Duration	400,000.00
Total Estimated Budget			**424,500.00**

7.2 Budget Justification

Hardware Costs

A **Raspberry Pi device** is used as a lightweight experimental hardware platform to simulate computing workloads and monitor system behavior during CI/CD pipeline executions. Raspberry Pi boards typically cost around **LKR 17,000–23,000 depending on the model** .

A **DHT22 temperature sensor** is used to measure heat generation during computational workloads, allowing the research to observe the relationship between processing activity and physical heat output. DHT22 sensors typically cost between **LKR 450 and LKR 1,280 in Sri Lankan electronics stores** .

Additional components such as a **microSD card, power supply, and cables** are required for running the monitoring system and storing monitoring data.

Cloud Services

Cloud infrastructure is required to run CI/CD pipelines, store monitoring data, and perform sustainability analysis. Oracle Cloud Infrastructure provides a **free tier with compute and storage resources**, which is sufficient for experimental deployment and prototype evaluation.

Software Licenses

The system relies on open-source technologies including **Python, psutil, pandas, and machine learning libraries**, which eliminates licensing costs. CI/CD tools such as **GitHub Actions and Jenkins** also provide free tiers suitable for academic experimentation.

Human Effort Cost

The human effort cost represents the estimated value of development and research work contributed by the **five-member project team**. This includes system design, implementation, testing, experimentation, and documentation activities over the project duration. The estimated value reflects the equivalent cost of entry-level software engineering effort for the project.

8. WORK BREAKDOWN STRUCTURE (WBS)

WBS ID	Task	Description	Timeline
M2.0	Project Planning	Initial planning and understanding of Green DevOps monitoring and sustainability measurement concepts	Jan 2026
M2.1	Literature Review	Study research papers on Green DevOps, CI/CD monitoring, energy estimation, and carbon-aware computing	Jan – Feb 2026
M2.2	Problem Definition	Identify the lack of sustainability monitoring in CI/CD pipelines and define research objectives	Feb 2026
M2.3	Proposal Preparation	Prepare research proposal including methodology, monitoring architecture, and dataset design	Late Feb – Early Mar 2026
M2.4	Proposal Presentation	Present research proposal to evaluation panel	16th – 18th Mar 2026
M2.5	Monitoring System Design	Design monitoring component architecture and sustainability dataset structure	Mar 2026
M2.6	Metric Collection Module Development	Develop scripts to collect CI/CD pipeline metrics such as CPU usage, memory usage, and execution duration	Apr – May 2026
M2.7	Energy Estimation Module	Implement proxy-based energy estimation model based on collected metrics	May – Jun 2026
M2.8	Carbon Estimation Integration	Integrate carbon intensity data sources such as ElectricityMaps API	Jun 2026
M2.9	Hardware Monitoring Setup	Configure Raspberry Pi and temperature sensor to observe system heat generation during workloads	Jun – Jul 2026
M3.0	Data Collection	Collect monitoring data from CI/CD pipeline executions and experimental workloads	Jul – Aug 2026
M3.1	Dataset Generation	Structure collected monitoring data into sustainability datasets	Aug 2026
M3.2	System Integration	Integrate monitoring outputs with other Green DevOps system components	Aug – Sep 2026
M3.3	Evaluation & Testing	Evaluate energy estimation accuracy and monitoring performance	Sep 2026
M3.4	System Optimization	Improve monitoring accuracy and performance	Sep 2026
M3.5	Documentation & Final Deliverables	Prepare final outputs including monitoring dataset and experimental results	Sep – Oct 2026
M3.6	Final Report Writing	Complete detailed documentation of the monitoring component	Sep – Oct 2026
M3.7	Final Presentation & Viva	Demonstrate monitoring system and research outcomes	12th – 14th Oct 2026
M3.8	Research Paper Preparation	Prepare research paper based on monitoring framework and results	Oct 2026

Table 2: Work Breakdown Structure of the Monitor Component

9. GANTT CHART

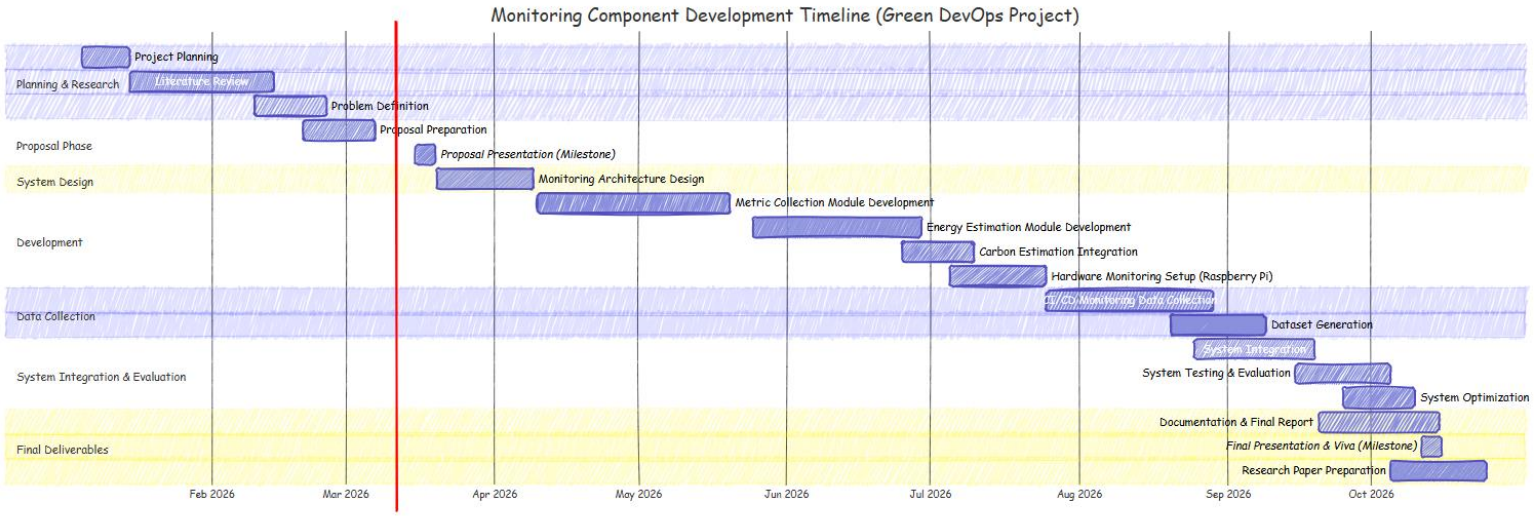


Figure 5: Gantt Chart of the Monitor Component

10. REFFERENCES LIST

- [1] M. Ailane, Y. Falah, and A. Hadi, “Green DevOps: A Strategic Framework for Sustainable Software Development,” Computer Sciences & Mathematics Forum, vol. 10, no. 1, 2025. Available: https://www.mdpi.com/2813-0324/10/1/5
- [2] H. Claßen, J. Thierfeldt, J. Tochman-Szewc, P. Wiesner, and O. Kao, “Carbon-Awareness in CI/CD,” arXiv preprint arXiv:2310.18718, 2023. Available: https://arxiv.org/abs/2310.18718
- [3] N. Saavedra, A. Mendes, and J. F. Ferreira, “Environmental Impact of CI/CD Pipelines,” arXiv preprint arXiv:2510.26413, 2025. Available: https://arxiv.org/abs/2510.26413
- [4] J. Perez et al., “Software Frugality in Continuous Integration: An Empirical Study,” arXiv preprint arXiv:2410.15816, 2024. Available: https://arxiv.org/abs/2410.15816
- [5] P. Aneggi et al., “Energy-Aware Continuous Integration Pipelines,” arXiv preprint arXiv:2602.12081, 2026. Available: https://arxiv.org/abs/2602.12081
- [6] A. Hindle, “Green Software Engineering: The Future of Sustainable Computing,” IEEE Software, vol. 30, no. 1, pp. 30–37, 2013. Available: https://ieeexplore.ieee.org/document/6511331
- [7] A. Beloglazov and R. Buyya, “Energy Efficient Resource Management in Virtualized Cloud Data Centers,” Future Generation Computer Systems, vol. 28, no. 5, pp. 755–768, 2012. Available: https://doi.org/10.1016/j.future.2011.04.017
- [8] International Energy Agency, “Data Centres and Data Transmission Networks,” 2023. Available: https://www.iea.org/reports/data-centres-and-data-transmission-networks
- [9] Green Software Foundation, “Software Carbon Intensity (SCI) Specification,” 2023. Available: https://sci.greensoftware.foundation
- [10] Cloud Carbon Footprint, “Open Source Cloud Emissions Monitoring Tool.” Available: https://www.cloudcarbonfootprint.org

11. APPENDICES

Appendix A – Monitoring Dataset Structure Example

Field Name	Description
pipeline_stage	Stage of CI/CD pipeline
execution_time	Time taken for pipeline stage
cpu_usage	CPU utilization percentage
memory_usage	Memory consumption
energy_consumption	Estimated energy usage
carbon_emission	Estimated carbon emissions
timestamp	Time of execution

Appendix B – Monitoring Script Example

Example snippet:

```
import psutil

cpu_usage = psutil.cpu_percent(interval=1)
memory_usage = psutil.virtual_memory().percent
print(cpu_usage, memory_usage)
```